

Design CampusEats — Services, Contracts & Schema

This document covers the capability list, the services and the members responsible for each one, the contract exposed by every service, the full specification of the system's central operation, and the validation of each service against the five service properties.

Task 1 — Capability List	Page 2
Services and Ownership	Page 4
Task 3 — Service Contracts	Page 5
Task 4 — Full Specification of placeOrder	Page 7
Task 6 — Service Property Validation	Page 9

Task 1: Capability List

1 Purpose

This task derives, from the Assignment 1 brief, the complete set of distinct jobs performed by the CampusEats system. Each job is treated as a Capability, expressed as a verb phrase, and forms the basis for identifying services, drawing service boundaries, and defining contracts. A Capability represents a coherent unit of behaviour offered to one or more actors, namely the student, the outlet staff member, the administrator, the delivery person, and in one case the system itself acting on their behalf. The assignment requires at least six Capabilities; nineteen are identified below, grouped into seven functional categories, so that later service boundaries can be drawn with minimal ambiguity.

2 Method

Each Capability was derived by pairing every actor named in the Who Uses It section with every relevant Noun and Verb described in the brief. Capabilities that appear as explicit Verbs in the brief are marked as directly stated. Capabilities that are necessary for the system to function but are only implied by the brief, most notably account creation and authentication, are marked as inferred and are justified individually in Section 3. Capabilities C11 and C17 through C19 are marked as inferred from the team's revised service design rather than from the brief itself, since the brief predates the decision to bring on-campus delivery into scope. For every Capability, the responsible actor, the Data Entities it reads or writes, and a formal description are recorded, so that each Capability can later be traced to exactly one owning service, since no two services may share data.

3 Assumptions Behind the Inferred Capabilities

The brief states in its Nouns section that a User is a person who has an account in the system and that a User carries one of three roles. It further states in its Who Uses It section that access to functionality depends on role, since a student, an outlet staff member, and an administrator are each permitted a different set of actions. A system with role dependent behaviour cannot function without a way to create an account, prove identity before each session, and maintain the details tied to that account. Registration, authentication, and profile maintenance are therefore treated as first class Capabilities in this document, on the same footing as the capabilities stated explicitly in the Verbs section, because Identity and Access Management typically forms its own bounded context once services are drawn, and omitting it at this stage would leave a gap in the benchmark that the service design depends on.

4 Capability Catalogue

ID	Capability	Actor	Data Entities	Description
A. Identity and Access Management (inferred)				
C1	Register a User Account	Student, Outlet Staff, Administrator	User	Creates a new User record with a role of student, staff, or administrator, capturing the credentials and profile information required for subsequent authentication. Staff accounts are created by an administrator under Capability C13 rather than through open self registration.
C2	Authenticate a User	Student, Outlet Staff, Administrator	User	Verifies a User's submitted credentials against the stored account and establishes an authenticated session, after which the system enforces access to every other Capability according to the User's assigned role.
C3	End a Session	Student, Outlet Staff, Administrator	User	Allows an authenticated User to explicitly terminate their current session, after which the credentials must be resubmitted through Capability C2 before any role restricted Capability can be used again.
C4	Maintain a User Profile	Student, Outlet Staff, Administrator	User	Allows an authenticated User to view and update the contact and profile details held in their own User record, independent of the role assigned to that record.
B. Catalogue Management (stated)				
C5	Browse Outlet Menu	Student	Outlet, Menu Item	Allows a student to view the food items offered by a chosen outlet, including the name, price, and availability of each Menu Item. This Capability is read only and does not modify system state.
C6	Manage Menu Item Availability	Outlet Staff	Menu Item	Allows an outlet staff member to mark a Menu Item as available or unavailable, preventing students from ordering items that are currently sold out.
C. Order Lifecycle (stated)				
C7	Place an Order	Student	Order, Order Item, Menu Item	Allows a student to select one or more Menu Items from a single outlet, specify a quantity for each item and a delivery address, and submit the collection as a new Order, which is created together with its Order Items and an initial status of placed.
C8	Track Order Status	Student	Order	Allows a student to retrieve the current status of a previously placed Order and observe its progression through the states placed, preparing, ready, and delivered.
C9	Update Order Status	Outlet Staff	Order	Allows an outlet staff member to advance the status of an Order through preparation, moving it from placed to preparing and from preparing to ready. The final transition to delivered is driven by Capability C18, once the linked Delivery completes.
D. Payment and Delivery (stated / inferred)				
C10	Pay for an Order	Student	Payment, Order	Allows the system to record that payment for a given Order has been completed, creating a Payment record associated with that Order, after

				confirming with the order that it exists and is not already paid.
C11	Create a Delivery for an Order	System	Delivery	When an order is placed, the system automatically creates a Delivery record using the delivery address supplied with the order, so that the order's fulfilment can be tracked from the outlet to the student separately from the order's own status.
E. Platform Administration (stated)				
C12	Register and Manage Outlets	Administrator	Outlet	Allows an administrator to add a new Outlet to the platform and to maintain its details, such as location and working hours, over time.
C13	Manage Staff Accounts	Administrator	User, Outlet	Allows an administrator to create staff User accounts and assign each staff member to a particular Outlet, establishing which staff member may act on behalf of which Outlet.
C14	View Platform Activity	Administrator	User, Order, Outlet	Allows an administrator to view overall activity across the platform, spanning all outlets, so that the system as a whole, rather than a single outlet, is being managed properly.
F. Notification (stated)				
C15	Send Order Notifications	System	Notification, Order	Allows the system to generate a Notification whenever an important event occurs, most notably a change in the status of an Order, and to address that Notification to the affected student or outlet staff member.
C16	Receive Notifications	Student, Outlet Staff	Notification	Allows a student or an outlet staff member to view the Notifications that have been generated on their behalf, most notably order status updates delivered by Capability C15.
G. Delivery Fulfilment (inferred)				
C17	Assign a Delivery Person	Administrator	Delivery Assignment, Delivery	Allows an administrator to assign a delivery person to a pending Delivery, creating a Delivery Assignment record that links the delivery person to that Delivery.
C18	Update Delivery Status	Delivery Person	Delivery	Allows the delivery person assigned to a Delivery to update its status as it progresses (for example, picked up, out for delivery, delivered). Once the Delivery reaches its final state, the linked Order's own status is advanced to match.
C19	Track Delivery Status	Student	Delivery	Allows a student to view the current status of the Delivery associated with their Order, separate from and in addition to the Order's own status from Capability C8.

5 Traceability to the Brief

Capabilities C1 through C4 are inferred from the account and role based access model described in the Nouns and Who Uses It sections, as justified in Section 3. Capabilities C5 through C10 correspond directly to actions available to the student and the outlet staff member during the browsing, ordering, and payment workflow. Capabilities C12 through C14 correspond to the administrative responsibilities described in the Who Uses It section, with C14 tracing to the statement that the administrator views the overall activity of the platform. Capabilities C15 and C16 together formalise the notification behaviour that the brief describes as a system wide concern rather than an action tied to a single actor. Capability C11 and Capabilities C17 through C19 are not drawn from the original brief; they reflect the team's updated service design, which brought on-campus delivery into scope after the brief was written, and are marked inferred for that reason rather than being justified against brief text.

Services and Ownership

The nineteen Capabilities from Task 1 are grouped into six services, each formed around the single Data Entity its capabilities touch. This is the benchmark diagram the team built to, together with the member responsible for implementing each service.

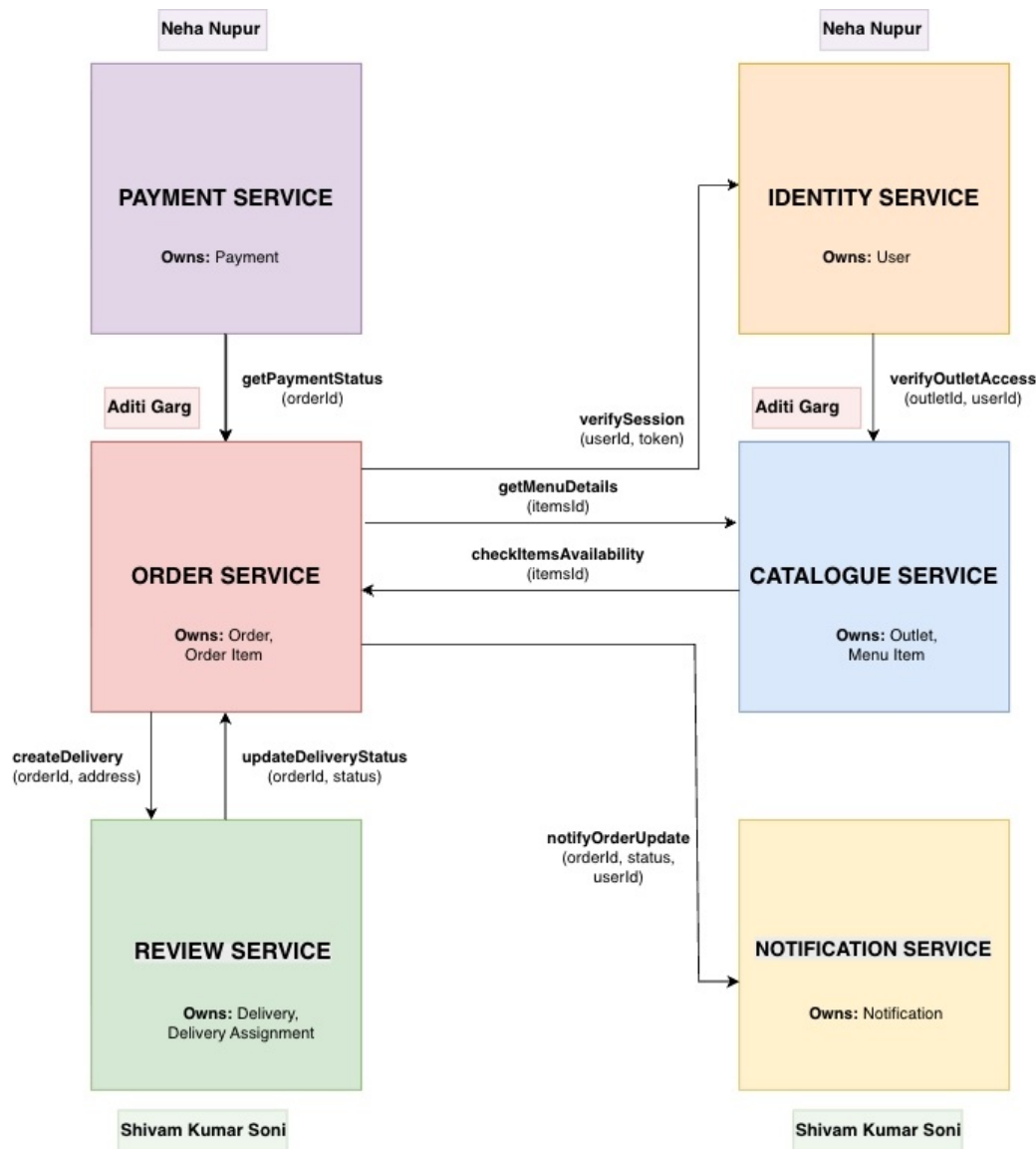


Figure 1. CampusEats service boundaries, cross-service calls, and the team member responsible for each service.

Service	Owns (Data)	Team Member	Capabilities It Implements (from Task 1)
Identity Service	User	Neha Nupur	C1 Register a User Account, C2 Authenticate a User, C3 End a Session, C4 Maintain a User Profile, and the account-creation half of C13
Catalogue Service	Outlet, Menu Item	Aditi Garg	C5 Browse Outlet Menu, C6 Manage Menu Item Availability, C12 Register and Manage Outlets, the outlet-linking half of C13
Order Service	Order, Order Item	Aditi Garg	C7 Place an Order, C8 Track Order Status, C9 Update Order Status, and the order-facing half of C14
Payment Service	Payment	Neha Nupur	C10 Pay for an Order
Review Service	Delivery, Delivery Assignment	Shivam Kumar Soni	C11 Create a Delivery for an Order, C17 Assign a Delivery Person, C18 Update Delivery Status, C19 Track Delivery Status
Notification Service	Notification	Shivam Kumar Soni	C15 Send Order Notifications, C16 Receive Notifications

Assumption. C13 (Manage Staff Accounts) and C14 (View Platform Activity) each touch more than one Data Entity, so neither fits inside a single service box. C13 is split: Identity Service creates the User record, and Catalogue Service confirms the outlet the staff member is being linked to; the two stay in sync by passing an outlet identifier as plain data between them, never by sharing a table. C14 is not drawn as an operation owned by any one service — it is an aggregate view built by an administrator-facing client that calls `listOrdersForOutlet`, `user` lookups, and `outlet` lookups separately and combines the results on the client side.

Task 3: Service Contracts

1 Purpose

This section defines the **contract** exposed by every service identified above. A contract lists only what a service promises to the outside world — the operation name, the input it needs, the output it returns, and the errors it can report. It never mentions how the service stores its data, what its tables look like, or what the internal identifiers are shaped like. The inside of each service stays hidden, exactly as the service boundaries require.

2 Contracts

Order Service and Catalogue Service are treated as the two main services, since together they carry the core CampusEats journey — a student browsing an outlet's menu and then placing an order against it — so each gets three or more operations below. The remaining four services get at least one operation each, as the task requires.

2.1 Order Service (main service)			
Operation	Input	Output	Errors
placeOrder	Calling student's identity, an outlet reference, a delivery address, and a list of menu item references each with a quantity	An order reference, its starting status, the line items priced out, and the total amount due	EmptyItemList OutletUnavailable ItemUnavailable InvalidQuantity InvalidDeliveryAddress
getOrderStatus	An order reference and the identity of whoever is asking	The order's current status and the times it moved through earlier statuses	OrderNotFound NotAuthorised
updateOrderStatus	An order reference, the identity or service calling, and the next status	The order's new status and the time it changed	OrderNotFound InvalidStatusTransition NotAuthorised
listOrdersForOutlet	An outlet reference and an optional status to filter by	A list of matching orders, each with its status and item summary	OutletNotFound NotAuthorised
getPaymentStatus	An order reference	Whether that order has already been marked paid, using Order Service's own record of it	OrderNotFound

updateOrderStatus is the one operation two different callers reach for two different reasons: an outlet staff member calls it directly to move an order from placed to preparing to ready, and Review Service calls it (labelled updateDeliveryStatus in Figure 1) to advance an order to delivered once its linked Delivery completes.

2.2 Catalogue Service (main service)			
Operation	Input	Output	Errors
getOutletMenu	An outlet reference	The outlet's name and hours, and its menu items with price and availability	OutletNotFound
getMenuDetails	A list of menu item references	For each item, its name, its current price, and the outlet it belongs to	ItemNotFound
checkItemsAvailability	An outlet reference and a list of menu item references with quantities	For every item requested, whether it belongs to that outlet and whether it is currently available	OutletNotFound ItemNotFound
verifyOutletAccess	An outlet reference and a user reference	Whether that outlet exists and that user is recorded as staff for it	OutletNotFound NotAuthorised
setItemAvailability	A menu item reference, the staff member's identity, and the new availability flag	The item's confirmed, updated availability	ItemNotFound NotAuthorised
registerOutlet	The administrator's identity and the new outlet's name, location, and hours	A reference to the newly created outlet	NotAuthorised ValidationError

getMenuDetails and checkItemsAvailability exist specifically so that Order Service can price and validate an incoming placeOrder request without ever reading the Menu Item table itself — see Task 4.

2.3 Identity Service			
Operation	Input	Output	Errors
authenticate	An email address and a password	A session token and the caller's role	InvalidCredentials
verifySession	A user reference and a session token	Whether the token is valid for that user, and the role attached to the session	InvalidSession SessionExpired
register	Name, email, password, and the	A reference to the newly created	DuplicateEmail ValidationError

	requested role	account	
--	----------------	---------	--

2.4 Payment Service

Operation	Input	Output	Errors
recordPayment	An order reference, the amount paid, and a payment method	A payment reference and its status	OrderNotFound AlreadyPaid AmountMismatch PaymentFailed

Before recording a new payment, Payment Service calls Order Service's `getPaymentStatus` to confirm the order exists and is not already marked paid — see Task 3, Section 3.

2.5 Review Service (owns Delivery, Delivery Assignment)

Operation	Input	Output	Errors
createDelivery	An order reference and a delivery address	A delivery reference and its starting status	OrderNotFound ValidationError
updateDeliveryProgress	A delivery reference, the delivery person's identity, and the new status	The delivery's updated status and the time it changed	DeliveryNotFound InvalidStatusTransition NotAuthorised
assignDeliveryPerson	A delivery reference and a delivery person's identity	A reference to the new assignment	DeliveryNotFound NotAuthorised
getDeliveryStatus	An order reference and the identity of whoever is asking	The delivery's current status and the times it moved through earlier statuses	DeliveryNotFound NotAuthorised

Once a delivery reaches its final status, Review Service calls Order Service's `updateOrderStatus` (labelled `updateDeliveryStatus` in Figure 1) so the order's own status advances to delivered.

2.6 Notification Service

Operation	Input	Output	Errors
notifyOrderUpdate	A recipient's identity, an order reference, and the order's new status	A reference to the created notification	RecipientNotFound
getNotifications	The caller's identity	A list of notifications addressed to that caller, newest first	NotAuthorised

3 Cross-Service Calls Implied by the Contracts

No two services share a table, so any operation that needs data owned by another service must call that service's contract instead of reaching into its storage. The contracts above create the following calling pattern, matching Figure 1 (with the one correction noted in Section 2.2), and carried forward into the internal walk-through of `placeOrder` in Task 4.

Caller	Calls	Reason
Order Service	Identity Service - <code>verifySession</code>	To confirm who is placing or tracking an order
Order Service	Catalogue Service - <code>getMenuDetails</code>	To fetch the current name and price of the items being ordered
Order Service	Catalogue Service - <code>checkItemsAvailability</code>	To confirm the items belong to the outlet and are available
Order Service	Review Service - <code>createDelivery</code>	To set up delivery tracking once an order has been accepted
Order Service	Notification Service - <code>notifyOrderUpdate</code>	To tell the student and the outlet staff whenever an order's status changes
Identity Service	Catalogue Service - <code>verifyOutletAccess</code>	To confirm the outlet reference is valid when linking a new staff account to it (C13)
Payment Service	Order Service - <code>getPaymentStatus</code>	To confirm the order exists and is not already paid, before recording a payment
Review Service	Order Service - <code>updateOrderStatus</code>	To advance the order to delivered once its linked delivery is completed

Task 4: Full Specification of placeOrder

1 Why placeOrder Is the Central Operation

Every other operation in the system either leads up to placeOrder (browsing the menu, authenticating) or follows from it (tracking status, paying, delivering). It is also the operation with the most work to do internally, since a single call has to touch data owned by three different services — Identity, Catalogue, and Order itself — while still keeping each service's storage private to that service. For this reason it is specified here in full, split clearly into what a caller is allowed to see and what stays hidden inside Order Service.

2 Public Contract

This is the only part of placeOrder that a caller — the student-facing client — ever needs to know about.

2.1 Input

Field	Meaning
Session token	Proves the caller is an authenticated student; not a data field the student fills in, but attached to every call
Outlet reference	Which outlet the order is being placed with
Delivery address	Where the order should be delivered; stored on the order itself and used to set up delivery tracking
Items	A non-empty list, each entry made up of a menu item reference and a quantity of one or more

2.2 Output (on success)

Field	Meaning
Order reference	Identifies the order for every later call — tracking, paying, delivering
Status	Always placed for a newly created order
Line items	Each requested item echoed back with the quantity and the price it was charged at
Total amount	The sum of all line items, computed by the system, never supplied by the caller
Placed at	The time the order was accepted

Delivery tracking is set up right after the order is written (Section 3, Step 6) but not synchronously as part of this response; a delivery reference becomes available a moment later through Review Service's getDeliveryStatus.

2.3 Error Cases

Error	When It Is Reported
NotAuthenticated	The session token is missing, invalid, or has expired
EmptyItemList	The item list has no entries
InvalidQuantity	Any item's quantity is zero or negative
InvalidDeliveryAddress	The delivery address is missing or empty
OutletNotFound	The outlet reference does not match any outlet
OutletClosed	The outlet is outside its stated working hours or has been deactivated
ItemNotFound	A requested menu item does not exist
ItemNotFromOutlet	A requested item exists but belongs to a different outlet than the one specified
ItemUnavailable	A requested item exists but is currently marked unavailable
CatalogueUnreachable	Catalogue Service could not be reached in time to validate the request

3 Internal Processing (Hidden from Callers)

Everything in this section belongs to Order Service alone. A caller never sees these steps, and no other service is allowed to depend on them staying this way — they can change freely as long as the public contract in Section 2 stays the same.

- Verify the session.** Order Service calls Identity Service's `verifySession` with the token attached to the request. If this fails, the call stops immediately with `NotAuthenticated`, before anything else runs.
- Check the shape of the request.** The item list must not be empty, every quantity must be a positive whole number, and the delivery address must not be empty. This is checked locally, without calling any other service.
- Validate and price against the catalogue.** Order Service calls Catalogue Service's `checkItemsAvailability` to confirm every item belongs to the outlet and is currently available, then calls `getMenuDetails` to fetch each item's current price. Order Service never queries the Menu Item data directly — it only ever asks Catalogue Service and trusts the answer.
- Price the order.** Using the prices returned in the previous step, Order Service computes a subtotal per line and a total for the whole order. The caller supplies quantities only; it never supplies or influences a price, which stops a student from placing an order at a price the outlet did not set.
- Create the order record.** Order Service writes a new Order together with its Order Items, inside its own storage, with the status

set to `placed` and the delivery address stored directly on the order. How this record is laid out, and what identifier scheme is used, is an internal detail with no bearing on the contract.

6. **Kick off delivery tracking.** Order Service calls Review Service's `createDelivery`, passing the order reference and delivery address. Like the notification step below, this call is allowed to fail without failing the order itself — the order has already been accepted by this point, and a missed delivery record is recovered separately rather than by rejecting the `placeOrder` call.
7. **Notify the outlet.** Order Service calls Notification Service's `notifyOrderUpdate` so the outlet's staff see the new order appear. This call is also non-blocking, for the same reason as Step 6.
8. **Return the result.** The order reference, status, priced line items, total, and timestamp are handed back to the caller, matching Section 2.2 exactly.

Task 6: Service Property Validation

1 Purpose

This task checks every one of the six services against five properties a well-designed service is expected to have: **reachable**, **self-contained**, **has a contract**, **independent**, and **loosely coupled**. The check is not a formality — four services in this design genuinely strain against the same property once the cross-service calls are followed through carefully, and this section says so plainly rather than marking every box as a pass. Where a property does not fully hold, a concrete fix is proposed that keeps the service boundary intact.

2 What Each Property Means Here

Property	What is being checked
Reachable	The service exposes its operations through a callable interface — another service or client can find it and invoke it. It is not logic buried inside another service's process.
Self-contained	The service owns and manages all the data it needs to do its job. No other service holds a piece of its state, and it does not reach into another service's storage to do its own work.
Has a contract	The service publishes an explicit interface — operation name, input, output, and errors — and callers rely on that interface rather than on internal details. This is the Task 3 deliverable.
Independent	The service can keep doing its job — be deployed, changed, and, most importantly, stay <i>up</i> — without being tied to another service's availability at the moment it is called.
Loosely coupled	Wherever the service touches another service, it does so only through that service's contract, never by depending on its internal schema, its storage, or its implementation choices.

Note on Independent vs. Loosely coupled. These two are easy to conflate but check different things. A service can be *loosely coupled* — every cross-service call goes through a published operation, never a shared table — while still failing to be *independent*, if that operation has to succeed synchronously, in real time, before the calling service can finish its own job. Order Service, below, is exactly this case: every call it makes is contract-based (loosely coupled, pass), but its central operation cannot complete unless two other services are up at that moment (independent, fail).

3 Validation Table

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled
Identity Service	Pass	Pass	Pass	Partial	Pass
Catalogue Service	Pass	Pass	Pass	Pass	Pass
Order Service	Pass	Pass	Pass	Fail	Pass
Payment Service	Pass	Pass	Pass	Partial	Pass
Review Service	Pass	Pass	Pass	Partial	Pass
Notification Service	Pass	Pass	Pass	Pass	Pass

Pass = the property fully holds. Partial = the property mostly holds, but one specific operation creates a real, narrow exception. Fail = the property does not hold for the service's central operation.

4 Per-Service Analysis

4.1 Identity Service

Reachable, Self-contained, Has a contract, Loosely coupled — all pass. Identity Service exposes `authenticate`, `verifySession`, and `register` as callable operations; it owns the `User` table alone and never reads another service's data to do its own work; and its one outbound call goes through Catalogue Service's published contract.

Independent — Partial. Every operation that matters most — authenticating a student, verifying a session, registering a student account — needs nothing from any other service and stays up regardless of what else is down. The one exception is the outlet-linking half of Capability C13 (Manage Staff Accounts): before Identity Service finalises a new staff account, it calls Catalogue Service's `verifyOutletAccess`. If Catalogue Service is unreachable at that moment, staff account creation cannot complete, even though nothing about that action touches Catalogue Service's data directly.

Fix. Do not block staff account creation on a synchronous outlet check. Create the `User` row immediately with `outlet_ref` set and an internal `outlet_verified = false` flag; verify the reference asynchronously (a retry job, or a message once Catalogue Service is reachable again) and flip the flag once confirmed. An administrator sees the account as "pending verification" in the rare case Catalogue Service is down, instead of the whole action failing outright.

4.2 Catalogue Service

All five properties pass without qualification. Catalogue Service is reachable through `getOutletMenu`, `getMenuDetails`, `checkItemsAvailability`, `verifyOutletAccess`, `setItemAvailability`, and `registerOutlet`; it owns `Outlet` and `MenuItem` alone; its contract is fully specified in Task 3; and — notably — it is the one service in this design that never calls out to any other service to do its own job.

Every one of its six operations can be completed using only its own tables. This makes it the cleanest example of all five properties holding together at once, and the one other services lean on rather than the reverse.

4.3 Order Service

Reachable, Self-contained, Has a contract, Loosely coupled — all pass. Order Service exposes five operations with a full contract (Task 3, Section 2.1); it owns Order and OrderItem alone, storing a priced snapshot and the delivery address directly rather than reaching into another service's table; and every outgoing call it makes goes through a published operation on another service's contract, never through shared storage.

Independent — Fail. This is the one outright failure in the design, and it sits on the operation that matters most. Walking through Task 4, Section 3: before `placeOrder` writes anything, it must successfully call Identity Service's `verifySession` (Step 1) and Catalogue Service's `checkItemsAvailability` and `getMenuDetails` (Step 3). If any of those services is unreachable, Order Service cannot accept a new order — not because of a bug, but because the design requires those answers before it is safe to write the order row. Order Service is loosely coupled (it never depends on how Identity or Catalogue store their data) but it is not independent (it cannot do its central job without both of them being up at that exact moment). The two calls Order Service makes after the order is already written — `createDelivery` and `notifyOrderUpdate` — are both non-blocking by design (Task 4, Section 3, Steps 6 and 7) and do not put `placeOrder` itself at risk. Tracking and listing operations are unaffected too, since neither calls another service — only placing a new order is exposed to this failure.

Fix. Trade a small amount of strictness for availability, in two places:

1. **Session check.** Let Order Service cache a session token as valid for a short window (for example, 60 seconds) after Identity Service last confirmed it, instead of calling `verifySession` on every single `placeOrder`. A brief Identity Service outage no longer blocks an order from a student who authenticated moments earlier.

2. **Catalogue check.** Let Catalogue Service publish price and availability changes (an event, or a periodic pull) into a small read-only cache inside Order Service. If Catalogue Service cannot be reached live, Order Service falls back to that cache, accepts the order with a `pending_confirmation` status instead of `placed`, and reconciles it automatically once Catalogue Service is reachable again, rather than rejecting a valid order outright.

Both fixes keep the contract and ownership boundary exactly as they are — Order Service still never reads Catalogue's or Identity's tables directly. They only change how strictly "must succeed right now" is enforced before a write is allowed.

4.4 Payment Service

Reachable, Self-contained, Has a contract, Loosely coupled — all pass. Payment Service exposes `recordPayment` with a full contract, owns the Payment table alone, and calls Order Service only through `getPaymentStatus`, never through Order Service's tables.

Independent — Partial. `recordPayment` calls Order Service's `getPaymentStatus` first, to confirm the order exists and is not already paid. If Order Service is unreachable, a payment cannot be recorded at that moment. This is the same class of issue as Order Service's own failure above, but narrower in effect: it blocks one operation on one service, not the platform's central transaction, so it is marked Partial rather than Fail.

Fix. Accept the payment immediately and store it with `status = pending` even if the call to Order Service's `getPaymentStatus` times out, then confirm the order reference asynchronously once Order Service responds. Losing a successful payment because of a momentary lookup failure is worse than briefly holding it in an unconfirmed state.

4.5 Review Service (delivery tracking)

Reachable, Self-contained, Has a contract, Loosely coupled — all pass. Review Service exposes `createDelivery`, `updateDeliveryProgress`, `assignDeliveryPerson`, and `getDeliveryStatus` with a full contract, owns Delivery and Delivery Assignment alone, and calls Order Service only through `updateOrderStatus`.

Independent — Partial, for the same structural reason as Payment Service: once a delivery reaches its final status, Review Service calls Order Service's `updateOrderStatus` to reflect that on the order itself. If Order Service is unreachable at that moment, the delivery person's own update to Review Service still succeeds — only the order's status lags behind until the call can be retried.

Fix. Let `updateDeliveryProgress` always succeed against Review Service's own Delivery table first, and treat the follow-up call to Order Service's `updateOrderStatus` as a retried, best-effort side effect — the same pattern already used for `createDelivery` and `notifyOrderUpdate` in Task 4. The delivery person is never blocked by Order Service's availability; only how quickly the order's own status catches up is affected.

4.6 Notification Service

All five properties pass without qualification. Notification Service exposes `notifyOrderUpdate` and `getNotifications` with a full contract, owns the Notification table alone, and — unlike every other service in this table except Catalogue Service — never calls out to another service to do its own job. It is only ever called by Order Service, and Task 4, Section 3, Step 7 explicitly makes that call non-blocking: if Notification Service is slow or unreachable, Order Service treats it as a recoverable side effect, not a reason to fail the order. Notification Service's own independence is therefore never at risk from its one caller's design.

5 Summary of Fixes

Service	Property Affected	Fix (does not change the service boundary or the Task 3 contract)
Identity Service	Independent (Partial)	Create staff accounts optimistically with an <code>outlet_verified</code> flag; verify the outlet reference asynchronously instead of blocking on it.
Order Service	Independent (Fail)	Cache recently verified sessions for a short window; cache Catalogue prices/availability and allow a <code>pending_confirmation</code> order status when Catalogue Service cannot be reached live.
Payment Service	Independent (Partial)	Accept the payment as <code>pending</code> and confirm the order reference asynchronously rather than blocking <code>recordPayment</code> on a live Order Service call.

Review Service	Independent (Partial)	Let a delivery status update always succeed locally first; retry the follow-up call to Order Service's updateOrderStatus as a best-effort side effect.
----------------	-----------------------	--